Knome & MPOEmployeeHub Platform

High-Level Architecture (HLA) & Federated SSO Integration Specification

Document Type	High-Level Architecture (HLA) Specification	**Version & Status**	**VERSION 1.0 — OFFICIAL APPROVED BASELINE**
Target Systems	Knome Enterprise Knowledge Platform & MPOEmployeeHub Central Identity	**Security & Session Model**	BFF Secure HttpOnly Cookie Boundary + YARP Gateway + Bearer JWT & RBAC

1. INTRODUCTION & EXECUTIVE CONTEXT

This document describes the High-Level Architecture (HLA) of the **Knome Enterprise Knowledge & Collaboration Platform** and its integration with **MPOEmployeeHub** — MPOnline Limited's Master Identity Provider, Single Sign-On (SSO) engine, and Enterprise Workforce Governance Authority.

1.1 Business Context & Strategic Objectives

MPOEmployeeHub and Knome deliver a 360-degree digital workspace for MPOnline Limited employees, structured across two core operational pillars:

MPOEmployeeHub (Master Identity & Governance)	
Enterprise SSO & Token Broker	Centralized authentication, OpenID Connect / OAuth2 token issuance
Master Identity & Lifecycle	Single source of truth for employee master data, status & credentials
Zero-Role Security Governance	Interception and assignment workflows for unassigned users
DPDP Act 2023 Compliance	Digital Personal Data Protection consent and privacy governance
Global Session Revocation	Centralized token blacklisting & session kill switch across all apps

Knome Platform (Knowledge & Social Collaboration)	
Social Activity Feed & Polls	LinkedIn-style real-time micro-posts, polls, mentions & reactions
Rich Technical Articles	Medium-style publishing, code syntax highlighting & rich media
Video Training & Playlists	YouTube-style learning repository, streams & departmental courses
Communities of Practice	Public and private domain groups with membership governance
Gamification & Recognition	Points engine, achievement badges, and enterprise leaderboards

1.2 ARCHITECTURE SCOPE & SUMMARY

The integrated ecosystem enforces a rigorous **Six-Layer Enterprise Architecture**. React provides browser presentation, Next.js / .NET provides the browser-facing BFF, YARP acts as the common security gateway, MPOEmployeeHub provides master identity & SSO, Knome Backend APIs provide knowledge business logic, and MSSQL Server provides persistence.

Layer	Layer Name	Primary Technology	Primary Architectural Responsibility
Layer 1	**React Frontend UI**	React 18 / Vite / TypeScript	Browser-facing user interface for Knome feeds, articles, videos & EmployeeHub SSO login. Zero token storage.
Layer 2	**Next.js / .NET BFF**	Next.js / ASP.NET Core BFF	**Backend-For-Frontend:** Manages browser session boundary, encrypted `HttpOnly` cookies & Cookie→Bearer token conversion.
Layer 3	**YARP Gateway (.NET)**	.NET / YARP Reverse Proxy	Common security & traffic gateway — SSL termination, URL routing, rate limiting, and security header enforcement.
Layer 4	**MPOEmployeeHub SSO**	ASP.NET Identity / RS256	Centralized Identity Provider. Owns master credential validation, Zero-Role evaluation, DPDP consent, and JWT token minting.
Layer 5	**Knome Backend APIs**	ASP.NET Core (.NET 10)	Modular business APIs owning Knome domains — Posts, Articles, Videos, Communities, Gamification, and Moderation.
Layer 6	**MSSQL Server**	Microsoft SQL Server 2022	Persistent data storage across dual isolated databases: `EmployeeHubDb` (Identity) and `Knome` (Business & Social).

2. SIX-LAYER PLATFORM WORKFLOW DIAGRAM

The diagram below illustrates the complete 6-layer platform workflow across **21 numbered execution steps**, covering the interaction between the React Frontend, Next.js BFF, YARP Gateway, MPOEmployeeHub SSO, Knome .NET APIs, and MS SQL Server, along with key authentication, token, API, callback, and database flows.

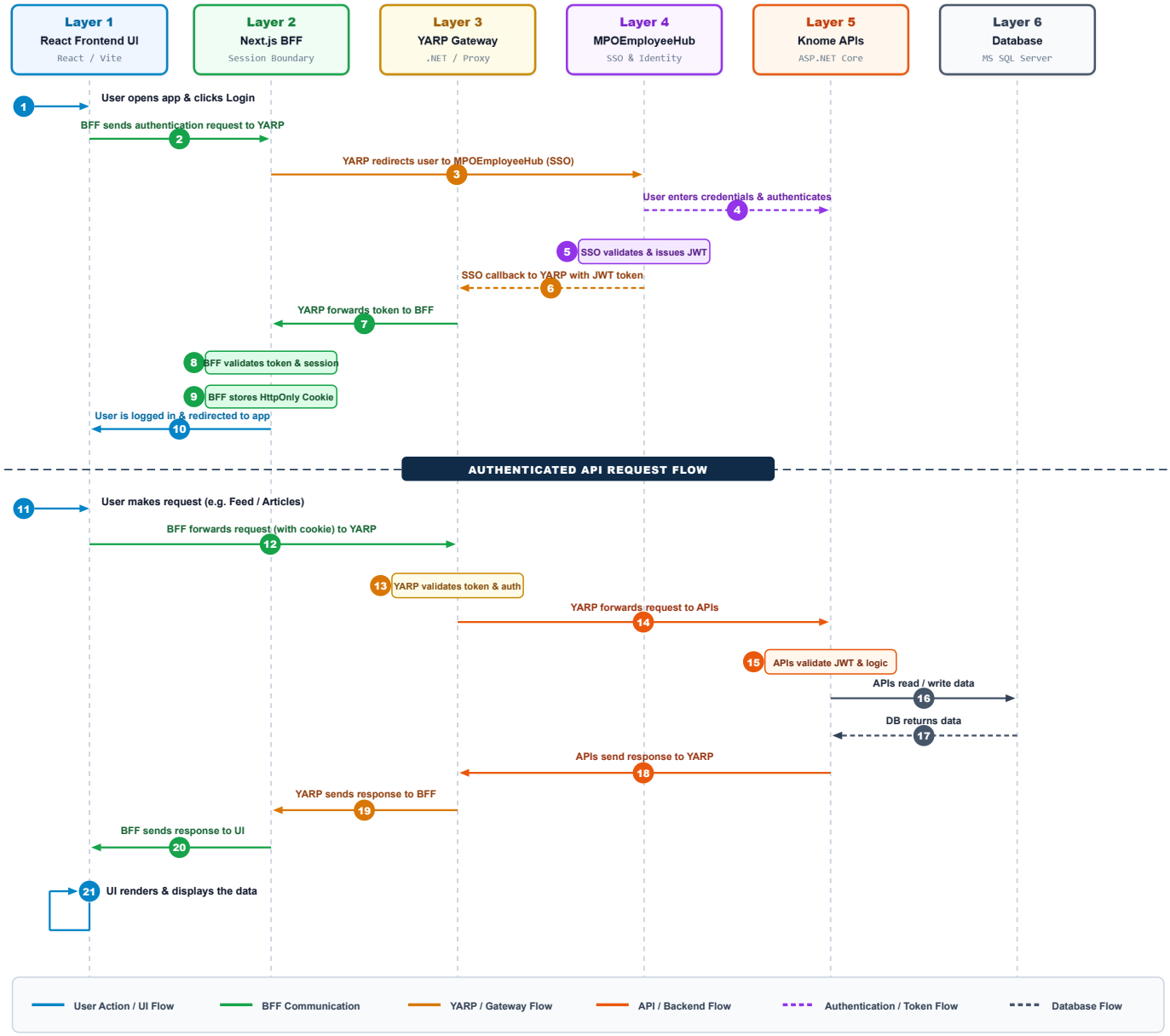

Figure 2.1: Knome & MPOEmployeeHub 6-Layer Platform Workflow Diagram

2.1 DETAILED 21-STEP WORKFLOW EXECUTION TABLE

The table below specifies the architectural execution details for each of the 21 steps shown in Figure 2.1 across the entire request/response lifecycle:

#	Source Layer	Target Layer	Architectural Execution Description
1	User / Browser	Layer 1 (React UI)	User opens Knome application URL (`https://knome.mponline.gov.in`) and enters SSO credentials.
2	Layer 1 (React UI)	Layer 2 (Next.js BFF)	React sends authentication request via client fetch to BFF proxy endpoint (`/api/auth/login`).
3	Layer 2 (Next.js BFF)	Layer 3 (YARP Gateway)	BFF forwards authentication request payload to centralized YARP security gateway.
4	Layer 3 (YARP Gateway)	Layer 4 (EmployeeHub SSO)	YARP routes auth request to EmployeeHub Layer 4 AuthService endpoint over internal TLS.
5	Layer 4 (EmployeeHub SSO)	Layer 6 (MSSQL Server)	EmployeeHub AuthService queries user master record, salt, BCrypt hash, and role status in `EmployeeHubDb` .
6	Layer 6 (MSSQL Server)	Layer 4 (EmployeeHub SSO)	Database returns employee master record, role associations, and DPDP consent flags.
7	Layer 4 (EmployeeHub SSO)	Layer 4 (Internal Engine)	Verifies password hash; checks Zero-Role status; signs standard RS256 JWT Token with claims & scopes.
8	Layer 4 (EmployeeHub SSO)	Layer 3 (YARP Gateway)	Returns signed JWT authorization token, refresh token, and sanitized employee profile DTO.
9	Layer 3 (YARP Gateway)	Layer 2 (Next.js BFF)	YARP forwards auth response and token stream to Next.js BFF.
10	Layer 2 (Next.js BFF)	Layer 2 (Internal)	BFF validates token signature against JWKS, creates server-side secure session envelope.
11	Layer 2 (Next.js BFF)	Layer 1 (React UI)	Sets secure `HttpOnly` session cookie in browser; returns sanitized user profile to UI.
12	Layer 1 (React UI)	User / Browser	User logged in; dashboard rendered with personalized Knome Feed, Articles & Communities.
13	User / Browser	Layer 1 (React UI)	User performs action in UI (e.g., views technical feed, creates post, uploads video).
14	Layer 1 (React UI)	Layer 2 (Next.js BFF)	React calls Next.js BFF API proxy endpoint with automatic browser cookie attachment.
15	Layer 2 (Next.js BFF)	Layer 3 (YARP Gateway)	BFF extracts session cookie and injects standard `Authorization: Bearer <JWT>` header.
16	Layer 3 (YARP Gateway)	Layer 3 (Policy Engine)	YARP enforces rate limiting, CORS policies, and validates token signature & claims.
17	Layer 3 (YARP Gateway)	Layer 5 (Knome APIs)	Authorized request forwarded to target Knome business controller (e.g., `PostController`).
18	Layer 5 (Knome APIs)	Layer 6 (MSSQL Server)	Knome API executes business logic, applies FluentValidation rules, reads/writes in `Knome` DB.
19	Layer 6 (MSSQL Server)	Layer 5 (Knome APIs)	MSSQL returns dataset recordsets or operation status to Knome API.
20	Layer 5 (Knome APIs)	Layer 3 (YARP Gateway)	Knome API returns standardized `ApiResponse<T>` JSON payload through gateway.
21	Layer 3 → 2 → 1	React Frontend UI	Gateway proxies to BFF, BFF strips internal headers, React UI renders updated state.

Summary of Core Invariants Across Steps 1–21

AUTHENTICATION INVARIANT

Steps 1–12 guarantee that the raw JWT token is handled exclusively between MPOEmployeeHub, YARP, and the BFF. The browser only stores an opaque `HttpOnly` cookie.

API EXECUTION INVARIANT

Steps 13–21 guarantee that all business logic and data access occur behind YARP's validation layer with structured error handling via `ApiResponse<T>` .

3. LOGIN & AUTHENTICATION WORKFLOW

MPOEmployeeHub is the sole owner of authentication, credentials, and master identity. The user's login request reaches EmployeeHub through the common YARP Gateway, which returns the callback through YARP to establish the BFF secure `HttpOnly` cookie session.

1. **User opens Knome in Browser:** User accesses application URL (`https://knome.mponline.gov.in`) in browser.
2. **React UI displays Login entry point:** React renders SSO login UI component and credential form.
3. **React sends auth request to Next.js BFF:** Client initiates authentication call `POST /api/auth/login` .
4. **BFF forwards auth request to YARP Gateway:** BFF passes request to common security gateway.
5. **YARP routes auth request to MPOEmployeeHub:** YARP proxies to Layer 4 Master AuthService over internal TLS.
6. **MPOEmployeeHub authenticates user:** Validates MPO ID & BCrypt password hash (work factor 11) against `EmployeeHubDb` .
7. **Zero-Role & Status Evaluation:** Checks active role status, account locking, and DPDP Act 2023 consent flags.
8. **MPOEmployeeHub creates & returns JWT token:** Issues signed RS256 authorization JWT token with claims, scopes & 30-day refresh token.
9. **MPOEmployeeHub redirects/callbacks via YARP:** Proxies auth callback result and token payload back to Next.js BFF.
10. **BFF processes authentication result:** BFF handles token verification via JWKS & creates server-side session envelope.
11. **Secure HttpOnly cookie session established:** Stores session token in `__Host-KnomeSession` `HttpOnly; Secure; SameSite=Strict` cookie.
12. **User reaches authenticated Knome app:** React renders authenticated portal dashboard with personalized feeds, articles, and groups.

[KEY ARCHITECTURE PRINCIPLES — LOGIN]

- **Zero Browser Token Exposure:** The browser never directly authenticates against MPOEmployeeHub or receives raw JWT tokens.
- **YARP Gateway Gatekeeping:** YARP is the common security gateway through which all authentication traffic passes.
- **Centralized Identity Authority:** The JWT / token lifecycle is owned entirely by MPOEmployeeHub — not by Knome UI or BFF.
- **Secure Session Boundary:** The BFF establishes an encrypted `HttpOnly` cookie to maintain the browser session securely.

JWT Claims Specification (Issued by MPOEmployeeHub)

```
{
  "iss": "https://employeehub.mponline.gov.in",
  "aud": "https://knome.mponline.gov.in",
  "sub": "EMP001",
  "name": "Rajesh Sharma",
  "email": "rajesh.sharma@mponline.gov.in",
  "department": "Technology Division",
  "designation": "Senior Software Engineer",
  "roles": ["Employee", "Community Admin"],
  "dpdp_consent": true,
  "exp": 1753100800
}
```

4. AUTHENTICATED FUNCTIONAL / API WORKFLOW

After login, all user-initiated actions follow this request-response cycle. Layer 5 (Knome Backend APIs) is responsible for serving all Knome business functionality. Authorization is enforced at YARP Gateway before any business API is reached.

4.1 Request Execution Path (User ➔ Database)

1. **User performs action in React UI:** User clicks button, creates post, publishes article, or requests video stream.
2. **React sends request to Next.js BFF:** UI calls BFF API proxy endpoint (`/api/proxy/posts`) with automatic cookie attachment.
3. **BFF forwards request to YARP Gateway:** BFF extracts session cookie and attaches `Authorization: Bearer <JWT>` header.
4. **YARP validates token & authorization:** Enforces gateway controls, rate limits, CORS policies, and validates JWT claims via JWKS.
5. **Authorized request reaches Knome API (Layer 5):** Routes to target business controller (e.g. `PostController` , `ArticleController`).
6. **Knome API executes business logic:** Applies FluentValidation, RBAC permissions check, domain logic & AutoMapper mapping.
7. **Knome API reads / writes in MSSQL Server:** Executes Entity Framework Core query, transaction, or stored procedure on `Knome` DB.

LAYER 5 — KNOME BACKEND API BUSINESS DOMAINS (ILLUSTRATIVE EXAMPLES)
Posts & Social Feed | Technical Articles & Publishing | Video Learning Hub | Communities of Practice
Gamification & Leaderboard | Enterprise Broadcasts | Content Moderation | Audit & Analytics

4.2 Response Path (Database ➔ User)

8. **MSSQL returns requested data to Knome API:** Database returns recordsets, affected rows, or transaction status.
9. **Knome API wraps payload & sends to YARP:** Transforms entity to DTO, wraps in `ApiResponse<T>` envelope, returns HTTP 200/201.
10. **Next.js BFF delivers sanitized data to React UI:** BFF proxies payload, strips internal headers; React UI updates TanStack Query cache & renders.

Standardized Response Envelope Contract

```
public class ApiResponse<T> {
    public bool Success { get; set; }
    public int StatusCode { get; set; }
    public string Message { get; set; }
    public T? Data { get; set; }
    public IEnumerable<string>? Errors { get; set; }
    public DateTime Timestamp { get; set; } = DateTime.UtcNow;
}
```

5. LOGOUT & SESSION INVALIDATION WORKFLOW

Logout is not simply hiding the UI. Authentication and session validity are invalidated at the identity layer (**MPOEmployeeHub**) and at the BFF session cookie layer. Once invalidated, the user's previous session cannot be used to resume protected operations.

1. **User clicks Logout in React UI:** User initiates sign-out action in Knome portal settings / header menu.
2. **React sends logout request to Next.js BFF:** UI calls BFF logout endpoint (`POST /api/auth/logout`).
3. **BFF sends logout request through YARP:** BFF proxies revocation call to centralized gateway.
4. **YARP routes logout request to MPOEmployeeHub API:** YARP routes logout to Identity Platform revocation endpoint.
5. **MPOEmployeeHub invalidates session & JWT token:** Revokes Refresh Token & registers JWT JTI in the central security blacklist.
6. **Logout callback sent back through YARP to BFF:** SSO confirms global session termination.
7. **BFF clears / expires browser HttpOnly cookie:** Destroys browser-facing secure session cookie (`Max-Age=0`).
8. **Protected access is no longer valid:** All subsequent API calls rejected with HTTP 401 Unauthorized at YARP / BFF.
9. **User is redirected back to Login page:** React navigates user to login entry screen; in-memory state cleared.
10. **Login lifecycle must start again for access:** User must authenticate from Step 1 for any new session.

[SECURITY STATEMENT — LOGOUT INVALIDATION]
After logout, the user's previous authenticated session / token cannot be used to continue protected Knome operations. Protected access is invalidated at MPOEmployeeHub Core (Refresh Token revoked & JTI blacklisted) and BFF cookie level (destroyed), forcing a complete new login lifecycle for subsequent access. Replay attacks with previous cookies are permanently blocked.

Global Session Revocation Matrix

Layer	Action on User Logout	Security Impact
Layer 1 (React UI)	Purges in-memory auth state, TanStack cache, redirects to `/login`	Client cannot display stale cached data.
Layer 2 (BFF)	Deletes server-side session, sets `Set-Cookie: __Host-KnomeSession=; Max-Age=0`	Browser cookie is wiped; cannot proxy any further calls.
Layer 3 (YARP)	Checks token revocation blacklist cache	Stolen raw JWTs are instantly rejected with HTTP 401.
Layer 4 (EmployeeHub)	Revokes Refresh Token in `EmployeeHubDb`	Tokens cannot be silently refreshed by any downstream app.

6. IMPLEMENTATION VIEW & ARCHITECTURE PRINCIPLES

6.1 Logical / Illustrative Monorepo Folder Structure

```
Knome-Platform/
├── Frontend/ (React 18 + Vite SPA) — knome-web/ (features: feed, articles, videos, communities, admin)
├── Backend/
│   ├── bff/ (Layer 2: Session Boundary) — Knome.Bff/ (BffAuthController.cs, BffProxyController.cs)
│   ├── gateway/ (Layer 3: Security Gateway) — Knome.Gateway/ (YARP Reverse Proxy, RateLimitingPolicy.cs)
│   └── services/
│       ├── EmployeeHub/ (Layer 4: IdP) — Domain, Contracts, Application, Infrastructure, API
│       └── Knome/ (Layer 5: Business APIs) — Domain, Contracts, Application, Infrastructure, API
```

6.2 TECHNOLOGY STACK SUMMARY

Component	Technology	Architectural Role & Operational Notes
Frontend UI	React 18 / Vite / TS	Browser-based SPA UI, component-driven dashboard, feed, articles & videos.
BFF Layer	Next.js / .NET BFF	Server-Side BFF, session handling, `HttpOnly` cookie boundary, token bridge.
API Gateway	.NET / YARP	Centralized reverse proxy, routing, rate limiting, security headers, SSL termination.
Identity Provider	MPOEmployeeHub SSO	Master identity platform, credential verification, RS256 JWT lifecycle & DPDP.
Business APIs	.NET 10 / ASP.NET Core	Modular microservice APIs owning Knome business functionality (Feeds, Videos).
Database	Microsoft SQL Server	Dual isolated DBs (`EmployeeHubDb` & `Knome`), ACID transactions & audit logs.

6.3 ARCHITECTURE PRINCIPLES & RULES

1. **Presentation Separation:** React is presentation only — strictly no direct database calls or business logic.
2. **BFF Session Boundary:** Next.js / .NET BFF is the browser session boundary — manages `HttpOnly` cookies and session storage.
3. **Gateway Gatekeeping:** YARP is the common security gateway through which all protected traffic passes.
4. **Master Identity Ownership:** MPOEmployeeHub owns master authentication, SSO, and identity token lifecycle end-to-end.
5. **Business Encapsulation:** Layer 5 owns all Knome business functionality — APIs, business rules, and integrations.
6. **Persistence Isolation:** Layer 6 owns persistent data storage — business data, transactions, and audit records in MSSQL Server.
7. **Encryption Standard:** All service-to-service communication must enforce HTTPS / TLS encryption without exception.
8. **Role & Compliance Validation:** Role-based authorization & DPDP compliance must be validated before protected operations execute.
9. **Complexity Encapsulation:** Backend complexity and raw JWT tokens must be hidden from browser client via BFF layer.
10. **LLD Alignment:** Exact low-level class designs, DB schemas, and API JSON payloads belong to Low-Level Design (LLD).